



School of Computer Science and Engineering

Faculty of Engineering

The University of New South Wales

Evaluation of Programming Languages and Compiler Systems for Verified LionsOS Components

by

Zhewen Shen

Thesis submitted as a requirement for the degree of
Bachelor of Advanced Computer Science (Honours)

Submitted: April 2025

Supervisor: Prof. Gernot Heiser

Student ID: z5388136

Abstract

This document describes the requirements to theses submitted for the Bachelor of Advanced Computer Science (Honours) degree at the School of Computer Science and Engineering. Requirements described are that of both of context and layout of the theses. The document is written using the L^AT_EX template provided by the school.

Acknowledgements

This work has been inspired by the labours of numerous academics in the Faculty of Engineering at UNSW who have endeavoured, over the years, to encourage students to present beautiful concepts using beautiful typography.

Further inspiration has come from Donald Knuth who designed $\text{\TeX}$, for typesetting technical (and non-technical) material with elegance and clarity; and from Leslie Lamport who contributed $\text{\LaTeX}$, which makes $\text{\TeX}$ usable by mortal engineers.

John Zaitseff, an honours student in CSE at the time, created the first version of the UNSW Thesis $\text{\LaTeX}$ class and the author of the current version is indebted to his work.

Abbreviations

PSOS Provably Secure Operating System

TCB Trusted Computing Base

UB Undefined Behaviour

Contents

1	Introduction	1
2	Background	4
2.1	seL4 Microkernel: A Foundation for Verified Systems	4
2.1.1	What is A Microkernel	4
2.1.2	The seL4 Microkernel	5
2.2	Building Verified Systems on seL4	7
2.2.1	CAMKES	7
2.2.2	The Microkit Framework	8
2.2.3	LionsOS	9
2.2.4	The Cost of Verification	10
2.3	The Compiler Trust Problem	10
2.3.1	Approaches to Removing the Compiler from the TCB	11
3	Related Work	14
3.1	Verification Friendly Languages	14
3.1.1	Rust in Particular	14
3.2	Verified Compilers	15
3.2.1	CompCert	15
3.2.2	CakeML	15

3.2.3	Pancake	16
3.2.4	VeLLVM	17
Bibliography		18

List of Figures

2.1	Monolithic (left) vs. Microkernel-based (right) Operating Systems (Heiser 2025)	5
2.2	Generations of Microkernels. Adapted from (Heiser 2024)	5
2.3	L4 microkernel family tree (Heiser et al. 2025).	6
2.4	Basic CAmkES Architecture (Trustworthy Systems 2025 <i>a</i>)	8
2.5	Microkit abstractions (Heiser 2025)	9
2.6	Translation validation proof chain (Heiser 2025).	12

List of Tables

Chapter 1

Introduction

In May 2017, the **WannaCry** ransomware outbreak dramatically underscored the fragility of modern operating systems. By exploiting an unpatched vulnerability in the Windows SMB protocol, WannaCry rapidly infected more than **230,000 computers across 150 countries**, crippling hospitals, businesses, and government services (National Institute of Standards and Technology 2017). This was not an isolated incident; it highlighted a broader trend of escalating security failures in widely used operating systems. Linux, Windows, and macOS — the foundational platforms for servers, desktops, and mobile devices — have each been plagued by hundreds even thousands of new vulnerabilities every year. In 2024 alone, Microsoft Windows 10/11 (and various other server editions) had 4939 reported CVE-listed vulnerabilities, the mainline Linux kernel had 3889, and Apple’s macOS had 519 (Stack.watch 2024). Many of these flaws are critical, enabling remote code execution or privilege escalation by attackers. The steady drumbeat of patches and security updates for all major OSes attests to an uncomfortable reality: **modern operating systems remain fundamentally insecure despite their importance.**

While this is tolerable in general-purpose computing, it is unacceptable in safety- and security-critical systems — such as those used in aircraft, medical devices, autonomous vehicles, and critical infrastructures — where a single failure can lead to catastrophic outcomes (Butler & Finelli 1993, Rushby 1997). These systems demand far stronger guarantees of correctness, isolation, and integrity than traditional OS architectures were ever designed to provide. In such domains, the operating system must not merely be “*secure enough*”; it must be provably correct.

This has motivated decades of research into the building of a **Provably Secure Operating System (PSOS)** (Neumann et al. 1975, Feiertag & Neumann 1979, Neumann et al. 1980, Neumann & Feiertag 2003) — systems for which critical properties can be mathematically verified. However, attempts to build such systems have historically been hindered by either impracticality or by excessive cost and complexity in the engineering and verification effort itself. That changed with the seL4 Microkernel (Klein et al. 2009), the first operating system kernel to be fully mathematically verified. seL4

provides strong spatial and temporal isolation, fine-grained access control via capabilities, and proofs of integrity, and confidentiality. The seL4 project demonstrates that formal verification can be applied to real-world systems of significant complexity.

The security guarantees provided by seL4 make it an attractive foundation for building a Provably Secure Operating System. And indeed, there have been some real-world deployments — including autonomous aircraft (Cofer et al. 2018), and, more recently, autonomous vehicles (Qu 2024). However, more than a decade after seL4’s verification, we still haven’t seen a complete operating system stack built on top of it that is itself formally verified — again owing to challenges in both the **engineering** and **verification** of the system.

To address these challenges, the seL4 Microkit (seL4 Foundation 2023) and LionsOS (Heiser et al. 2025) were developed, demonstrating that it is possible to build a performant and practical system on top of seL4 without incurring significant engineering overhead. However, while these tools and design philosophies solve many of the engineering difficulties, the problem of verification beyond the kernel still requires further studies.

LionsOS aims to enable end-to-end verification of security and safety enforcement. Achieving this requires that components within its Trusted Computing Base (TCB) are not only functionally correct but formally verified down to the executable code. In seL4’s verification story, the C compiler was removed from the TCB through translation validation (Sewell et al. 2013) — a fragile and laborious process that imposes limitations on code complexity. A more robust approach is to use programming languages with well-defined semantics and a verified compiler that produce *correct* binaries by construction.

This thesis aims to investigate three such programming languages and their compiler stacks for building a **provably secure** LionsOS without compromises to **performance** or **practicality**:

1. **CompCert (Leroy 2009)**: a verified optimising compiler for the CLight dialect of C
2. **CakeML (Kumar et al. 2014)**: a functional language with a end-to-end verified compiler
3. **Pancake (Pohjola et al. 2023)**: a low-level systems language that builds upon CakeML’s compiler backend.

We will be providing a detailed, quantitative and qualitative analysis of each language system — examining their suitability for different LionsOS components, the productivity impact on both developers and verifiers, and the performance of the generated code.

Structure Overview

The remainder of this thesis will be structured as follows:

Add structure overview once the other sections are done

Chapter 2

Background

Add a brief introduction to what this chapter will be talking about... etc...

2.1 seL4 Microkernel: A Foundation for Verified Systems

This section provides an overview of microkernel architecture as a foundation for building high-assurance systems. We begin by discussing the core principles and evolution of microkernel design, followed by an overview of seL4 (Klein et al. 2009) as a representative third-generation microkernel. Finally, we outline the engineering challenges that arise when building a general-purpose operating systems on top of seL4.

2.1.1 What is A Microkernel

A microkernel is a minimalist approach to operating system kernel design. Unlike a monolithic design, which places all core operating system services – such as device drivers, file systems, and networking – within the kernel, a microkernel includes only the most essential components, typically low-level address space management, thread scheduling, and inter-process communication (IPC) (Härtig et al. 1997).

The remaining OS functionality runs in user space as separate, isolated services (Härtig et al. 1997). These components communicate via the aforementioned IPC mechanism. This separation improves modularity, security, and stability, as faults in individual services are less likely to compromise the entire system. However, typical microkernel-based systems often faced performance challenges due to the overhead of IPC and context switching (Chen et al. 2024).

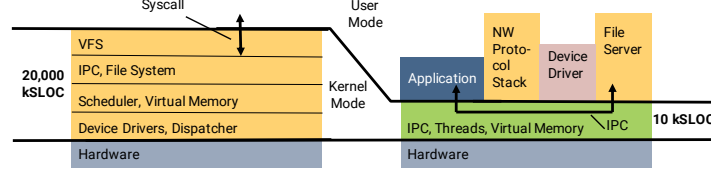


Figure 2.1: Monolithic (left) vs. Microkernel-based (right) Operating Systems (Heiser 2025)

Evolution of Microkernel Design

Microkernel architectures have evolved over time in response to early limitations and shifting system requirements. Figure 2.2 illustrates the three major generations of microkernel design (Heiser 2024). First-generation microkernels retained much of the functionality typical of monolithic kernels, resulting in many system calls and extremely poor IPC performance. In response, second-generation microkernels embraced minimality as a core principle, stripping unnecessary functionality from the kernel to achieve greater simplicity and significantly improved performance. Third-generation microkernels, such as seL4, further refined this approach by focusing not only on performance but also on formal assurance, including functional correctness and strong isolation guarantees. They achieve this by aggressively minimising the kernel and delegating responsibilities such as memory management to user space.

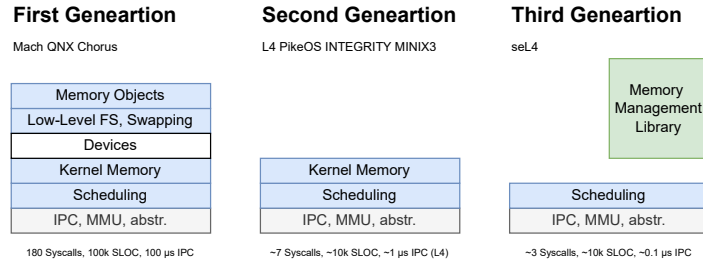


Figure 2.2: Generations of Microkernels. Adapted from (Heiser 2024)

2.1.2 The seL4 Microkernel

seL4 is a third-generation microkernel in the L4 family (Figure 2.3), tracing its lineage to Liedtke’s work on high-performance kernels in the mid 1990s (Liedtke 1995). Like its predecessors, seL4 adopts the minimality principle and provides only the essential mechanisms required to securely abstract hardware (Heiser et al. 2022).

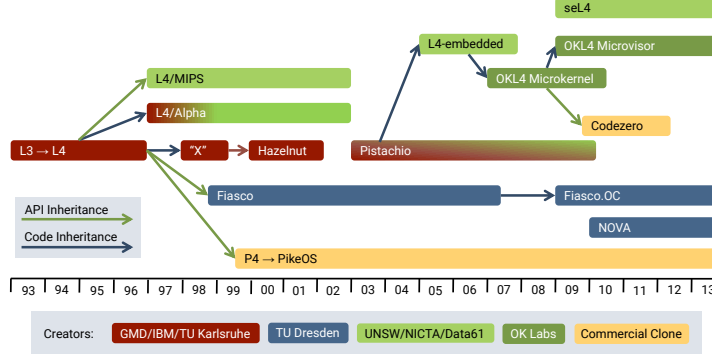


Figure 2.3: L4 microkernel family tree (Heiser et al. 2025).

seL4 combines this minimalist design with strong formal verification: it is the only operating system kernel with machine-checked proofs of functional correctness, integrity enforcement, and confidentiality down to the executable binary (Klein et al. 2014). Central to seL4’s security model is its capability-based access control, where capabilities are unforgeable tokens granting specific rights to kernel objects. This model enables fine-grained control and strict spatial isolation between components, ensuring that faults remain contained and cannot compromise the wider system.

Despite the additional proof infrastructure, empirical studies have established seL4 as the world’s fastest microkernel with performance within 25% of the limits imposed by hardware it runs on Mi et al. (2019). These properties make seL4 an attractive foundation for safety- and security-critical systems that require strong isolation, such as autonomous vehicles, medical devices, and industrial control systems. Real-world deployments have demonstrated its potential, including use in autonomous military aircraft (Cofer et al. 2018) and commercial electric vehicles (Qu 2024). However, such adoptions remain relatively rare and confined to highly specialised domains, reflecting the considerable engineering challenges associated with building practical and general-purpose systems directly on top of seL4’s low-level API.

Challenges Beyond the Kernel

While seL4 provides unparalleled assurance and strong isolation properties, its minimal design exposes developers directly to low-level mechanisms that are typically hidden in conventional operating systems. Core responsibilities such as memory management, including page table construction and management of kernel memory regions, are delegated entirely to user space (Elkaduwe et al. 2008). Although this delegation is critical for enabling provable isolation, it significantly increases the complexity of system construction. Developers must not only reason carefully about the allocation and protection of resources, but must also implement substantial infrastructure before a functional system can even be developed.

As a result, seL4 has often been described as the “*assembly language of operating sys-*

tems” (Heiser et al. 2025) – a platform offering fine-grained, powerful primitives, but requiring deep expertise and meticulous engineering to use correctly. For most industrial developers, especially those targeting safety- and security-critical domains, this level of complexity presents a substantial barrier to adoption. Projects that attempt to deploy seL4 at scale without substantial framework support frequently encounter unsustainable engineering overheads, and several early seL4-based initiatives were abandoned after a few years due to these challenges (Heiser et al. 2025). This has motivated several initiatives aiming to address the practical barriers of building systems on seL4, including CAMkES (Kuz et al. 2007) (Section 2.2.1), Microkit (seL4 Foundation 2023) (Section 2.2.2), and LionsOS (Heiser et al. 2025) (Section 2.2.3).

2.2 Building Verified Systems on seL4

Thus far, we have provided a high-level overview of what a microkernel-based operating system looks like, and discussed the challenges involved in building practical systems directly on seL4. In this section, we introduce two key frameworks, CAMkES and Microkit, which aim to reduce the engineering effort required to construct systems on top of seL4. Finally, we present LionsOS, a state-of-the-art operating system built using Microkit, designed to balance security, performance, and practicality.

2.2.1 CAMkES

CAMkES (Component Architecture for microkernel-based Embedded Systems) is a software development and runtime framework designed to simplify the construction of complex multiserver systems on top of the seL4 microkernel. It follows a component-based software engineering approach (see Figure 2.4), where a system is modelled as a set of isolated software components with explicitly defined interaction interfaces and connections (Kuz et al. 2007, seL4 Foundation 2025).

CAMkES provides a language for describing component interfaces, components themselves, and the overall system architecture. A build tool processes these descriptions and automatically combines developer-provided component code with generated scaffolding and glue code to produce a complete, bootable system image.

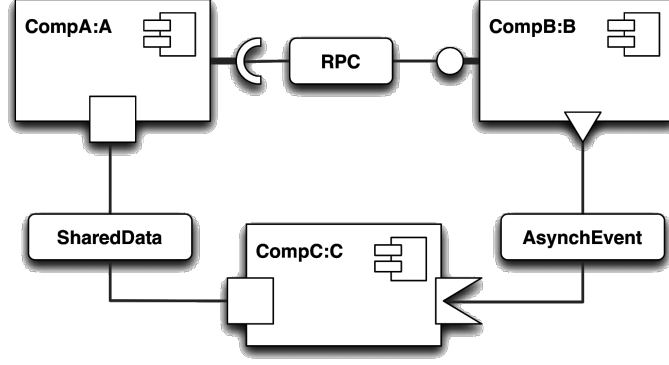


Figure 2.4: Basic CAMkES Architecture (Trustworthy Systems 2025a)

However, CAMkES’s abstractions come at a cost. Its component model introduces additional complexity, and its lack of a dedicated software development kit (SDK) results in a cumbersome build process that is tightly coupled to the already complex kernel build system. Furthermore, the generated scaffolding introduces notable runtime overheads (Heiser 2025), which can be problematic for performance-sensitive embedded systems. These limitations have motivated the development of more lightweight alternatives, such as Microkit, which aim to simplify system construction while preserving seL4’s strong assurance and performance.

2.2.2 The Microkit Framework

Microkit was developed as a lightweight framework and software development kit (SDK) for building statically-architecture systems on seL4. It provides a minimal set of simple abstractions that closely mirror seL4’s core mechanisms, enabling developers to construct modular systems with minimal overhead and improved efficiency (Trustworthy Systems 2025b, seL4 Foundation 2023).

The central abstraction in Microkit is the Protection Domain (PD), which bundles an address space (VSpace), access rights (CSpace), a thread and scheduling context, and basic synchronisation primitives. A Protection Domain serves as the unit of execution and isolation and is associated with executable code and a scheduling priority. Communication between PDs occurs through user-defined Communication Channels (CCs), which support event notifications and protected procedure calls (PPC). Shared memory communication is also supported through memory objects.

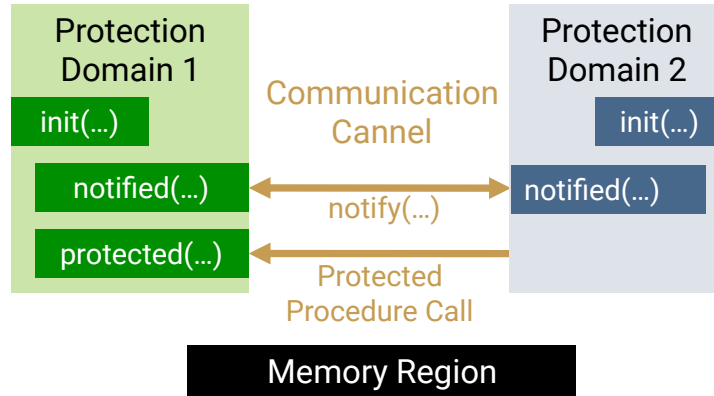


Figure 2.5: Microkit abstractions (Heiser 2025)

By intentionally exposing only a small number of abstractions, Microkit encourages developers to design systems that naturally align with seL4’s principles of isolation and minimality. Its communication model further promotes event-driven designs, as Protection Domains are typically structured around asynchronous notifications triggered by Communication Channels. This approach to simplicity not only reduces engineering complexity but also allows formal verification of system behaviour more tractable.

2.2.3 LionsOS

LionsOS is a modular operating system designed for safety- and security-critical embedded systems, built on top of the seL4 microkernel and the Microkit framework. It aims to combine the formal assurance and isolation properties of seL4 with a practical and performant system architecture that supports real-world deployment (Heiser et al. 2025).

The design of LionsOS is guided by the principles of strict separation of concerns, least privilege, and *radical simplicity*, following the KISS (“Keep It Simple, Stupid”) principle. Each component is narrowly scoped and operates independently, enabling fine-grained fault isolation and laying the groundwork for scalable formal verification. Rather than attempting to support highly dynamic general-purpose workloads, LionsOS focuses on static system architectures common to embedded and cyberphysical systems, where resource allocation is known ahead of time.

Another key design principle of LionsOS is the use of use-case-specific policies instead of attempting to generalise resource management and scheduling policies across diverse applications. This approach significantly reduces complexity compared to conventional general-purpose OS designs and allows tailored optimisations without sacrificing assurance.

In contrast to earlier attempts at modular, microkernel-based operating systems that suffered severe performance penalties due to high IPC and context-switching costs

(Chen et al. 2024), LionsOS demonstrates that a simple and minimalist approach to OS design can deliver competitive or even superior performance. Empirical evaluation shows that LionsOS exceeds the throughput and latency performance of Linux in network-heavy workloads despite its highly modular architecture (Heiser et al. 2025).

2.2.4 The Cost of Verification

While LionsOS addresses many of the engineering and performance challenges traditionally associated with microkernel-based systems, the issue of formal verification remains a major barrier to achieving a provably secure operating system with a complete end-to-end verification story.

The experience of verifying seL4 demonstrated that functional correctness of real-world operating systems can be formally proven. However, the verification cost was substantial: verifying approximately 8,500 lines of C code required around 12 person-years of work, with an estimated cost of \$350 per line of code (Klein et al. 2009). Although such an investment is justified for a operating system kernel, it would be impractical for verifying an entire operating system, which is significantly larger and much more complex.

Recent developments in automated verification have shown that significant reductions in verification effort are possible by leveraging SMT-based techniques instead of traditional interactive theorem proving (ITP). Approaches based on symbolic state-space exploration, guided by heuristics, allow many proof obligations to be discharged automatically without human intervention. These techniques are particularly effective when applied to small, simple modules – an advantage made possible by LionsOS’s minimalistic and modular system design.

However, even assuming that source-level verification becomes significantly more tractable through automation, a fundamental obstacle remains: ensuring that the compiled binary faithfully implements the verified source code. Without addressing the trustworthiness of the compiler, the overall system cannot achieve true end-to-end assurance.

In section 2.3, we will discuss the implications of having the compiler as part of the trusted computing base, namely the challenges posed by unsafe language semantics, compiler optimisations, miscompilations, and the difficulty of ensuring that verified source-level properties are preserved at the binary level. We then examine approaches for addressing these issues through translation validation and verified compilation.

2.3 The Compiler Trust Problem

In his 1984 Turing Award Lecture, “Reflections on Trusting Trust,” Ken Thompson revealed a disturbing possibility: even a fully open-source system can have undetectable

backdoors if the compiler used to build it is compromised (Thompson 1984). He described how a maliciously modified compiler could inject a backdoor into an operating system’s `login` binary and, more insidiously, propagate this backdoor into future compiler binaries – even if the backdoor no longer appears in any source code. This “trusting trust” attack shows that no amount of source-level verification can guarantee system integrity if the compiler itself cannot be trusted.

Even putting aside malicious attacks, compilers are large and complex software systems, and like all such systems, they inevitably contain bugs. Despite decades of engineering effort, miscompilations – cases where a correct source program is silently translated into an incorrect binary – remain a recurring problem across all major compiler ecosystems. A large-scale study by Yang et al. (2011) found 325 previously unknown bugs across widely used C compilers, including GCC, Clang, and Intel CC, many leading directly to silent miscompilations.

To illustrate the severity of this issue, Listing 2.1 shows a real-world miscompilation that was introduced in the optimization phase of GCC 9.1¹, where the program would return 0 with no optimisations and 1 with optimisations. It remained unfixed until GCC 12.

```

1 int main(void) {
2     for (unsigned a = 0, b = 0; a < 6; a += 1, b += 2)
3         if (b < a)
4             return 1;
5     return 0;
6 }
```

Listing 2.1: Example of a miscompilation bug in GCC (Bug #102666)

For a provably secure system like **LionsOS**, this problem is not merely theoretical. A single miscompilation, whether due to a malicious injection, a compiler bug, or an unsound optimisation, can undermine the entire verification effort. Since proofs are typically constructed at the source level, the correctness of the resulting binary hinges entirely on the correctness of the compilation process itself. As such, we require techniques that remove the compiler from the TCB and therefore eliminate the need to trust it.

2.3.1 Approaches to Removing the Compiler from the TCB

Translation Validation

In seL4’s verification story, the compiler was removed from the TCB via a method known as translation validation, where for each individual compilation, the resulting

¹https://gcc.gnu.org/bugzilla/show_bug.cgi?id=102666

binary is formally shown to refine the verified C source program (Sewell et al. 2013).

As shown in Figure 2.6, the binary is lifted into a control-flow graph representation through a verified disassembler within HOL4, based on a formal ISA model (such as ARM or RISC-V). The C program, already formalised in Isabelle/HOL, is translated into the same graph language through semantics-preserving transformations. To account for compiler optimisations, additional rewrite rules are applied at the graph level within the theorem prover.

The equivalence between the two programs is then checked by splitting them into small chunks and discharging semantic preservation queries to SMT solvers. In theory, establishing full program equivalence is undecidable, as it reduces to the halting problem. In practice, however, the compiler’s transformations are sufficiently regular to make the problem tractable for real-world code. Even so, the process remains fragile: minor changes to the source code can alter the structure of these SMT chunks and cause proof attempts to fail. In addition, the toolchain used in the seL4’s approach places practical limitations on the complexity of the source code.

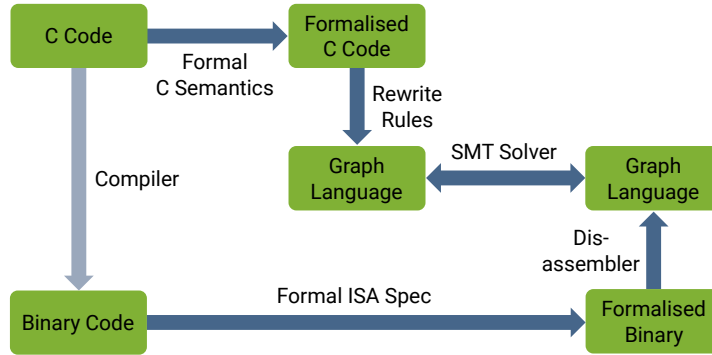


Figure 2.6: Translation validation proof chain (Heiser 2025).

Verified Compilers

A much better alternative to translation validation is the use of a *verified compiler*, where it is guaranteed by construction that the generated binary preserves the semantics of the source program. Rather than verifying each individual compilation after the fact, the correctness of every compilation is a direct consequence of the compiler’s proof.

Although compilers are large and intricate pieces of software, the property they must establish is surprisingly simple: the produced binary must refine the semantics of the input program. Formally, this can often be phrased as a simulation or refinement relation between the source and target languages, typically proved inductively on the structure of the compilation steps. In summary, every transformation applied by the compiler must be proven to preserve or refine the program’s observable behaviour. Once this invariant is established for all stages, the compiler can be trusted for all programs

within its specification (Leroy 2009). This approach eliminates the fragility associated with translation validation and removes the compiler from the TCB entirely.

We will be providing more details on three production-grade verified compilers – CompCert, Pancake, and CakeML – in the Related Works section.

Chapter 3

Related Work

3.1 Verification Friendly Languages

TODO: Introduce the trend toward verification-friendly languages (SPARK Ada, F*, Rust, Frama C?), highlight that while they improve source-level safety and verifiability, their unverified compilers still leave a large trusted computing base (TCB). Emphasize that full end-to-end system verification requires not just safer languages but verified compilation. Originally this section was going to be just talking about Rust (most likely just a subsection in the background) but following Peter's feedbacks from the seminar, I think it is needed to either make the distinction here or in the background section.

3.1.1 Rust in Particular

3.2 Verified Compilers

3.2.1 CompCert

CompCert is one of the first practical, production-grade verified optimising compilers for a substantial subset of the C programming language. Its correctness proof is mechanised in Coq and formally guarantees that the compiled assembly code behaves as specified by the semantics of the source C program. The verification covers the back-end optimisations and code generation phases. However, the front-end parsing and elaboration steps—which handle unsupported or ambiguous C constructs—are written in OCaml and are not verified. As a result, bugs found in the unverified front-end, as identified in (Yang et al. 2011), remain a residual risk.

Being a rather mature compiler, CompCert C supports most of ISO C99 and a significant portion of C11 (Leroy 2009). Given its target domain of safety- and mission-critical systems, language coverage and compatibility have never been a major concern. In our experience, the engineering overhead associated with adopting CompCert is relatively low compared to alternatives.

Despite prioritising verification over aggressive optimisation, CompCert produces code that is highly competitive with traditional compilers. On PowerPC and ARM platforms, CompCert’s generated code achieves about 90% of the performance of GCC 4 compiled at optimisation level `-O1`, and approximately 80–85% compared to `-O2` and `-O3`. The performance gap is primarily due to the absence of sophisticated loop transformations and high-level optimisations. Nonetheless, for most embedded and critical applications, this tradeoff between performance and assurance is well justified (Leroy 2009).

3.2.2 CakeML

CakeML is a verified functional programming language and toolchain based on a substantial subset of Standard ML (SML). It features a fully mechanised semantics in higher-order logic (HOL) and a proven-correct compiler capable of bootstrapping itself (Kumar et al. 2014). The language design is kept simple and functional to make formal verification tractable, although the standard basis library is still evolving.

The CakeML compiler consists of two frontends and a verified backend. The first frontend is a proof-producing translator that synthesises CakeML programs from HOL definitions, while the second is a conventional parser and type inferencer, both formally proved sound and complete. The backend generates machine code for several architectures, including x86, ARM, and RISC-V, and passes through multiple verified intermediate representations. The entire toolchain has been bootstrapped inside HOL, resulting in a verified binary that provably implements the compiler itself.

CakeML has been used to build a variety of verified applications, including Unix-like utilities (e.g., `grep`, `sort`, `diff`), proof checkers for certificates and SAT solvers, and larger systems like Candle, a verified re-implementation of the HOL Light theorem prover. It has also served as a basis for verified compilers targeting other languages, such as PureCake (for a lazy functional language) and Pancake (for low-level systems programming).

3.2.3 Pancake

Pancake is a recently developed verified systems programming language targeting low-level systems code, particularly device drivers (Pohjola et al. 2023). Designed to sit at an abstraction level between C and assembly. Currently, Pancake prioritises ease of verification over language features, deliberately eschewing sophisticated type systems to simplify formal reasoning.

The language design is radically simple. Pancake supports only three kinds of data: machine words, code labels (function pointers), and structs (bundles of machine words and labels).¹ There is no heap allocation at runtime (only statically allocated heap memory), no stack inspection, no concurrency primitives, and no undefined behaviour. Expressions are side-effect free, and evaluation order is fully specified, eliminating many sources of verification complexity common in C.

Verifying Device Drivers with Pancake

Pancake has been used to verify a performant Ethernet device driver for the Lions operating system. The driver targets a 1GbE NIC on an ARM-based SoC, and achieves near line-rate performance with only around 10% CPU overhead compared to an unverified C implementation (Zhao et al. 2025).

Verification combines two mechanisms: (1) the aforementioned Pancake compilation pipeline, and (2) an automated deductive verification frontend based on Viper. Pancake source code is annotated with function contracts and loop invariants, which are preserved through transpilation into Viper’s intermediate language for SMT-based verification. Memory safety, device interface compliance, network protocol correctness, and data integrity across packet transfers are all formally verified.

The implementation and verification for the entire ethernet driver took around three person-months of work by a systems engineer with no prior verification experience. The verification establishes end-to-end properties of the compiled binary, with the only major trusted components being the HOL4 theorem prover and the Viper verification framework.

¹Not C-style structs.

This result demonstrates that Pancake is capable of supporting low-level, performant, verifiable device drivers without introducing excessive engineering or verification overhead.

3.2.4 VeLLVM

came across this randomly, not sure if I should include it

VeLLVM (Verified LLVM) is a framework for mechanised reasoning about the LLVM intermediate representation (IR) and its transformations, formalised in Coq (Zhao et al. 2012). It provides a formal semantics for a subset of LLVM IR, along with verified proofs about specific transformation passes, such as memory safety enforcement.

However, VeLLVM does not constitute a verified compiler. It verifies transformations over an intermediate representation, rather than providing end-to-end correctness from source code to binary, and relies on an unverified backend.

As such, VeLLVM is not considered further in this thesis.

Bibliography

- Butler, R. W. & Finelli, G. B. (1993), ‘The infeasibility of quantifying the reliability of life-critical real-time software’, *IEEE Transactions on Software Engineering* **19**(1), 3–12.
- Chen, H., Miao, X., Jia, N., Wang, N., Li, Y., Liu, N., Liu, Y., Wang, F., Huang, Q., Li, K., Yang, H., Wang, H., Yin, J., Peng, Y. & Xu, F. (2024), Microkernel goes general: Performance and compatibility in the HongMeng production microkernel, *in* ‘18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)’, USENIX Association, Santa Clara, CA, pp. 465–485.
URL: <https://www.usenix.org/conference/osdi24/presentation/chen-haibo>
- Cofer, D., Gacek, A., Backes, J., Whalen, M., Pike, L., Foltzer, A., Podhradsky, M., Klein, G., Kuz, I., Andronick, J., Heiser, G. & Stuart, D. (2018), ‘A formal approach to constructing secure air vehicle software’, *IEEE Computer* **51**, 14–23.
- Elkaduwe, D., Derrin, P. & Elphinstone, K. (2008), Kernel design for isolation and assurance of physical memory, *in* ‘Proceedings of the 1st Workshop on Isolation and Integration in Embedded Systems’, IIES ’08, Association for Computing Machinery, New York, NY, USA, p. 35–40.
URL: <https://doi.org/10.1145/1435458.1435465>
- Feiertag, R. J. & Neumann, P. G. (1979), The foundations of a provably secure operating system (psos), *in* ‘Proceedings of the National Computer Conference’, AFIPS Press, pp. 329–334.
URL: <https://www.csl.sri.com/neumann/psos.pdf>
- Härtig, H., Hohmuth, M., Liedtke, J., Schönberg, S. & Wolter, J. (1997), ‘The performance of μ -kernel-based systems’, *SIGOPS Oper. Syst. Rev.* **31**(5), 66–77.
URL: <https://doi.org/10.1145/269005.266660>
- Heiser, G. (2024), ‘Comp9242 advanced operating systems: Microkernel design & implementation – the 25-year quest for the right api (week 05 part 1)’, <https://cgi.cse.unsw.edu.au/~cs9242/24/lectures/05a-uk.pdf>. Lecture slides, UNSW School of Computer Science & Engineering.
- Heiser, G. (2025), The sel4 microkernel: An introduction, Technical Report Revision 1.4, sel4 Project. Accessed: 2025-04-25.
URL: <https://sel4.systems/About/seL4-whitepaper.pdf>

- Heiser, G., Parker, L., Chubb, P., Velickovic, I. & Leslie, B. (2022), Can we put the “s” into iot?, in ‘2022 IEEE 8th World Forum on Internet of Things (WF-IoT)’, pp. 1–6.
- Heiser, G., Velickovic, I., Chubb, P., Joshy, A., Ganesh, A., Nguyen, B., Li, C., Darville, C., Zhu, G., Archer, J., Zhou, J., Winter, K., Parker, L., Duchniewicz, S. & Bai, T. (2025), ‘Fast, secure, adaptable: Lionsos design, implementation and performance’.
URL: <https://arxiv.org/abs/2501.06234>
- Klein, G., Andronick, J., Elphinstone, K., Murray, T., Sewell, T., Kolanski, R. & Heiser, G. (2014), ‘Comprehensive formal verification of an os microkernel’, *ACM Trans. Comput. Syst.* **32**(1).
URL: <https://doi.org/10.1145/2560537>
- Klein, G., Elphinstone, K., Heiser, G., Andronick, J., Cock, D., Derrin, P., Elkaduwe, D., Engelhardt, K., Kolanski, R., Norrish, M., Sewell, T., Tuch, H. & Winwood, S. (2009), sel4: formal verification of an os kernel, in ‘Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles’, SOSP ’09, Association for Computing Machinery, New York, NY, USA, p. 207–220.
URL: <https://doi.org/10.1145/1629575.1629596>
- Kumar, R., Myreen, M. O., Norrish, M. & Owens, S. (2014), ‘Cakeml: a verified implementation of ml’, *SIGPLAN Not.* **49**(1), 179–191.
URL: <https://doi.org/10.1145/2578855.2535841>
- Kuz, I., Liu, Y., Gorton, I. & Heiser, G. (2007), ‘Camkes: A component model for secure microkernel-based embedded systems’, *Journal of Systems and Software* **80**(5), 687–699. Component-Based Software Engineering of Trustworthy Embedded Systems.
URL: <https://www.sciencedirect.com/science/article/pii/S016412120600224X>
- Leroy, X. (2009), ‘Formal verification of a realistic compiler’, *Commun. ACM* **52**(7), 107–115.
URL: <https://doi.org/10.1145/1538788.1538814>
- Liedtke, J. (1995), ‘On micro-kernel construction’, *SIGOPS Oper. Syst. Rev.* **29**(5), 237–250.
URL: <https://doi.org/10.1145/224057.224075>
- Mi, Z., Li, D., Yang, Z., Wang, X. & Chen, H. (2019), Skybridge: Fast and secure inter-process communication for microkernels, in ‘Proceedings of the Fourteenth EuroSys Conference 2019’, EuroSys ’19, Association for Computing Machinery, New York, NY, USA.
URL: <https://doi.org/10.1145/3302424.3303946>
- National Institute of Standards and Technology (2017), ‘Bolstering government cybersecurity: Lessons learned from wannacry’. Accessed: 2025-04-19.
URL: <https://www.nist.gov/speech-testimony/bolstering-government-cybersecurity-lessons-learned-wannacry>
- Neumann, P. G., Boyer, R. S., Bressler, W. J., Boulware, D. W., Levitt, K. N., Feiertag, R. J., Robinson, L. & Heck, M. (1975), A provably secure operating system, Technical

- Report ESD-TR-75-372, SRI International, Computer Science Laboratory. <https://csrc.nist.gov/publications/history/neum75.pdf>.
- Neumann, P. G., Boyer, R. S., Feiertag, R. J., Levitt, K. N. & Robinson, L. (1980), A provably secure operating system: The system, its applications, and proofs, Technical Report CSL-116, SRI International, Computer Science Laboratory.
URL: <https://www.csl.sri.com/neumann/psos/psos80.pdf>
- Neumann, P. G. & Feiertag, R. J. (2003), PSOS Revisited , *in* ‘Computer Security Applications Conference, Annual’, IEEE Computer Society, Los Alamitos, CA, USA, p. 208.
URL: <https://doi.ieeecomputersociety.org/10.1109/CSAC.2003.1254326>
- Pohjola, J. r., Syeda, H. T., Tanaka, M., Winter, K., Sau, T. W., Nott, B., Ung, T. T., McLaughlin, C., Seassau, R., Myreen, M. O., Norrish, M. & Heiser, G. (2023), Pancake: Verified systems programming made sweeter, *in* ‘Proceedings of the 12th Workshop on Programming Languages and Operating Systems’, PLOS ’23, Association for Computing Machinery, New York, NY, USA, p. 1–9.
URL: <https://doi.org/10.1145/3623759.3624544>
- Qu, N. (2024), ‘seL4 in software-defined vehicles: Vision, roadmap, and impact at NIO’. Keynote at the 6th seL4 Summit.
URL: <https://sel4.systems/Foundation/Summit/2024/slides/software-defined.pdf>
- Rushby, J. (1997), Formal methods and their role in the certification of critical systems, *in* R. Shaw, ed., ‘Safety and Reliability of Software Based Systems’, Springer London, London, pp. 1–42. Also discusses DO-178B standards for certification in aviation software systems.
- seL4 Foundation (2023), ‘seL4 microkit GitHub’.
URL: <https://github.com/seL4/microkit>
- seL4 Foundation (2025), *CAMkES: Component Architecture for microkernel-based Embedded Systems*, seL4 Foundation. Accessed: 2025-04-26.
URL: <https://docs.sel4.systems/projects/camkes/>
- Sewell, T. A. L., Myreen, M. O. & Klein, G. (2013), Translation validation for a verified os kernel, *in* ‘Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation’, PLDI ’13, Association for Computing Machinery, New York, NY, USA, p. 471–482.
URL: <https://doi.org/10.1145/2491956.2462183>
- Stack.watch (2024), ‘2024 security vulnerability statistics’. Accessed: 2025-04-19.
URL: <https://stack.watch/stats/2024/>
- Thompson, K. (1984), ‘Reflections on trusting trust’, *Commun. ACM* **27**(8), 761–763.
URL: <https://doi.org/10.1145/358198.358210>
- Trustworthy Systems (2025a), ‘Camkes: Component architecture for microkernel-based embedded systems’, <https://trustworthy.systems/projects/OLD/camkes>. Accessed: 2025-04-26.

Trustworthy Systems (2025b), ‘The sel4 microkit’, <https://trustworthy.systems/projects/microkit/>. Accessed: 2025-04-26.

Yang, X., Chen, Y., Eide, E. & Regehr, J. (2011), Finding and understanding bugs in c compilers, *in* ‘Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation’, PLDI ’11, Association for Computing Machinery, New York, NY, USA, p. 283–294.

URL: <https://doi.org/10.1145/1993498.1993532>

Zhao, J., Legnani, A., Ung, T. T., Truong, H., Sau, T. W., Tanaka, M., Åman Pohjola, J., Sewell, T., Sison, R., Syeda, H., Myreen, M., Norrish, M. & Heiser, G. (2025), ‘Verifying device drivers with pancake’.

URL: <https://arxiv.org/abs/2501.08249>

Zhao, J., Nagarakatte, S., Martin, M. M. & Zdancewic, S. (2012), Formalizing the llvm intermediate representation for verified program transformations, *in* ‘Proceedings of the 39th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages’, POPL ’12, Association for Computing Machinery, New York, NY, USA, p. 427–440.

URL: <https://doi.org/10.1145/2103656.2103709>